

lab3

任务介绍

本次实验在实验1，2实现的语法树的基础上，实现中间代码生成，有两种可选形式，本人选择的是Zero IR

功能实现

总体来说，实现的是：

1. 控制流图 (`control_flow.hpp/cpp` , `cfg_builder.hpp/cpp`)

- 将线性的IR指令组织成结构化的块和函数
- 处理函数调用和返回值传递
- 区分全局变量和函数定义

2. IR指令系统 (`ir.hpp`)

- 定义了各种IR指令类型
- 每种IR指令都有特定的格式和语义，遵循Zero IR的规范

3. IR翻译器 (`ir_translator.hpp/cpp`)

- 为不同类型的AST节点提供专门的翻译方法
- 生成临时变量和标签来表示计算过程
- 将AST节点翻译成对应的IR指令序列

实现过程如下：

1. 拉取lab3，合并模版
2. 测试通过新增后版本的原有功能
3. 配置调试中间代码插件
4. 测试最小中间代码生成以及正确性
5. `ir.hpp`中添加一些基本类别：DEC,If,Load,Store,Param,Global,GetAddr
6. 支持基本表达式翻译（算数运算、函数调用、变量访问）
7. 支持条件语句(if-else)和逻辑表达式(&&, ||)的短路求值
8. 支持数组访问（基础实现）
9. 支持函数定义和参数传递

10. 改进的基本块划分，为后续优化提供基础（测试时暂不采用）
11. 进行覆盖性测试，修复函数参数列表翻译时创建别名的问题，改为查符号表创建
12. 修改checkFuncDef处函数参数命名，明确的强制统一命名以避免问题
13. 修改load,store类，拓展添加支持变量偏移量的功能
14. 实现对简单数组访问的支持
15. 区分数组的常量索引和变量索引访问：偏移量处理
 - a. 当数组索引是常量时（如 `a[5]`），直接计算出字节偏移量（索引值*4）
 - b. 变量索引处理：当数组索引是变量时（如 `a[i]`），计算出总偏移量，再与原地址相加，再取地址
16. 支持一维数组的各种初始化
17. 支持全局变量的声明，初始化（包括数组和int）
18. 支持多维数组的声明和基本初始化
19. 支持多维数组常量索引
20. 修正命令格式：变量索引先求值再取址
21. 修正命令格式：多个单目运算符先全部求值再返回
22. 修正命令格式：if中的表达式先求值再返回
23. 修改命令格式：函数参数先求值再传入
24. 添加支持：支持数组作为函数参数
25. 添加支持：局部多维数组复杂初始化
26. 添加支持：全局多维数组复杂初始化
27. 修正命令格式：将三种一元运算符处理成其他一般指令（`-a == 0-a !a` 用条件指令转换）
28. build_single_func中区分有无返回值的函数行为
29. 再次完善关于数组传参的处理：
 - 无索引
 - 索引数<数组维数
 - 索引数 = 数组维数

本次实验的迭代次数要多于前两次实验，大概是因为本次实验涉及输出格式的问题，大概多花了10次左右的迭代在等价转换输出格式以符合Zero IR的规范上。

实现中的亮点

emmm,本次实验基本都是面向语法规则+测试去做的，总体来说比较平滑，没有什么特别的亮点。数组相关的仍然是这次实验的难点，不过有了实验2的处理经验，也算大同小异。

另外一个重点是区分全局变量，不同于前两次实验，本次实验需要严格的区分和单独处理全局变量和局部变量，因为全局变量的地址以及初始化的处理都与局部变量不同。

最后就是仔细看IR语法，将一些不支持的如单元运算符!, -, +全部转换成支持的基本语句，如+可以直接丢，`-a<==>0-a`，`!a` 等价于：

```
1  T1 = a
2  T2 = #0
3  T3 = #1
4  IF T1 == T2 GOTO L1
5  T4 = T2
6  GOTO L2
7  LABEL L1:
8  T4 = T3
9  LABEL L2:
```

AI工具与代码借鉴使用声明

本实验本人没有借鉴任何除统一提供的模版之外的代码。

本报告没有使用AI工具编写。

本实验的代码部分由本人进行构思和调试以及设计性编写，而部分结构重复的翻译工作由AI（github copilot）生成。

建议和想法

我觉得本次实验的指导书和模版仍然非常的全面和友好，有效地平滑了实验的难度曲线，感谢助教为此做出的贡献！